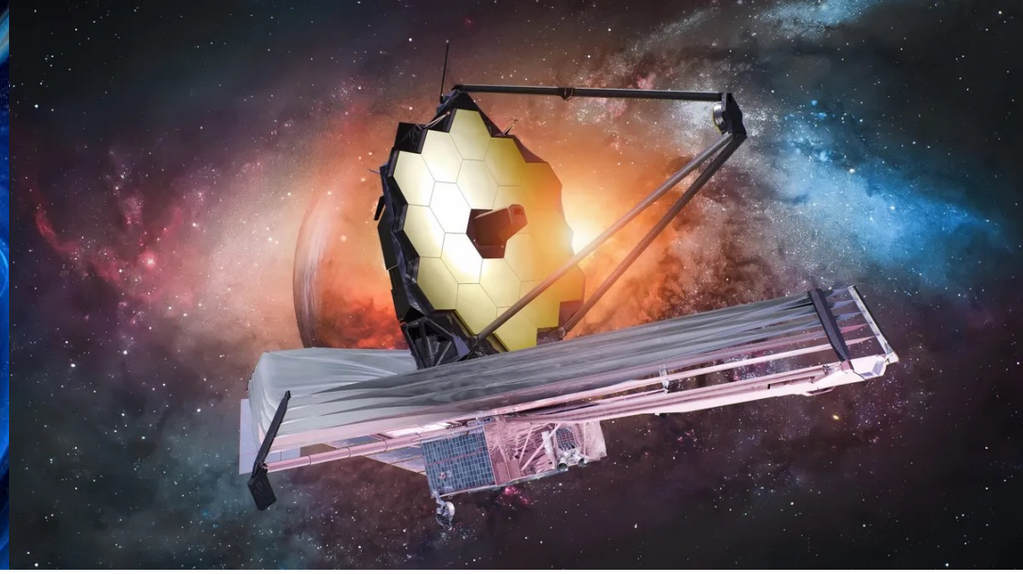
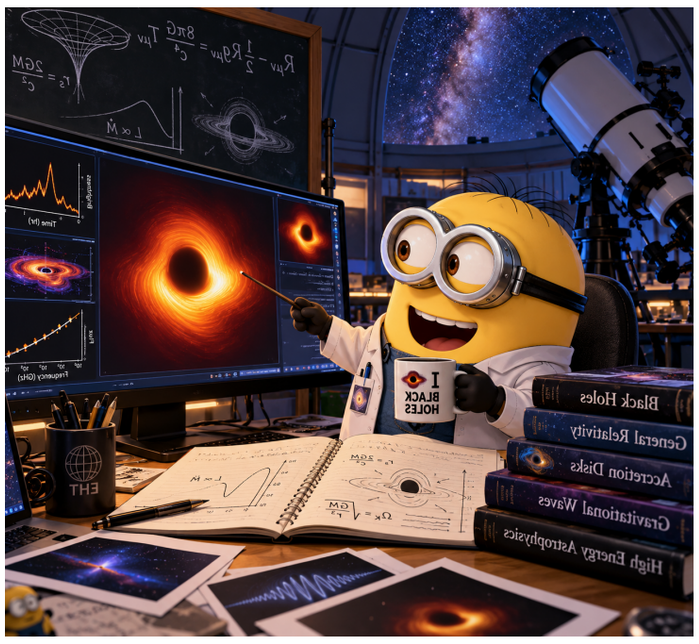
ABC for HPC

Martin Topinka — AÚUK + **RSE** @ MFF UK

Prague, 27/11/2026



- **Astronomy & Astrophysics**
  high energy transients, simulations, data science, data analysis pipelines, Machine Learning
  (Gamma-Ray Bursts, magnetars, Fast Radio Bursts)



- **Research Software Engineering (RSE)**
  "*To bridge the gap between the researchers and computers…*"
  (from HPC to AI and LLMs)
  http://rse.cuni.cz , http://hpc.cuni.cz

High Performance Computing

- Introduction HPC
- Introduction to Chimera cluster
- SLURM scheduler, submitting jobs
- Parallel programming
  — openMP (shared memory)
  — MPI (distributed memory)
- GPU
  — openACC
  — CUDA
  — Python on GPU
- * Bonus track — LLM MCP

There was a free lunch…

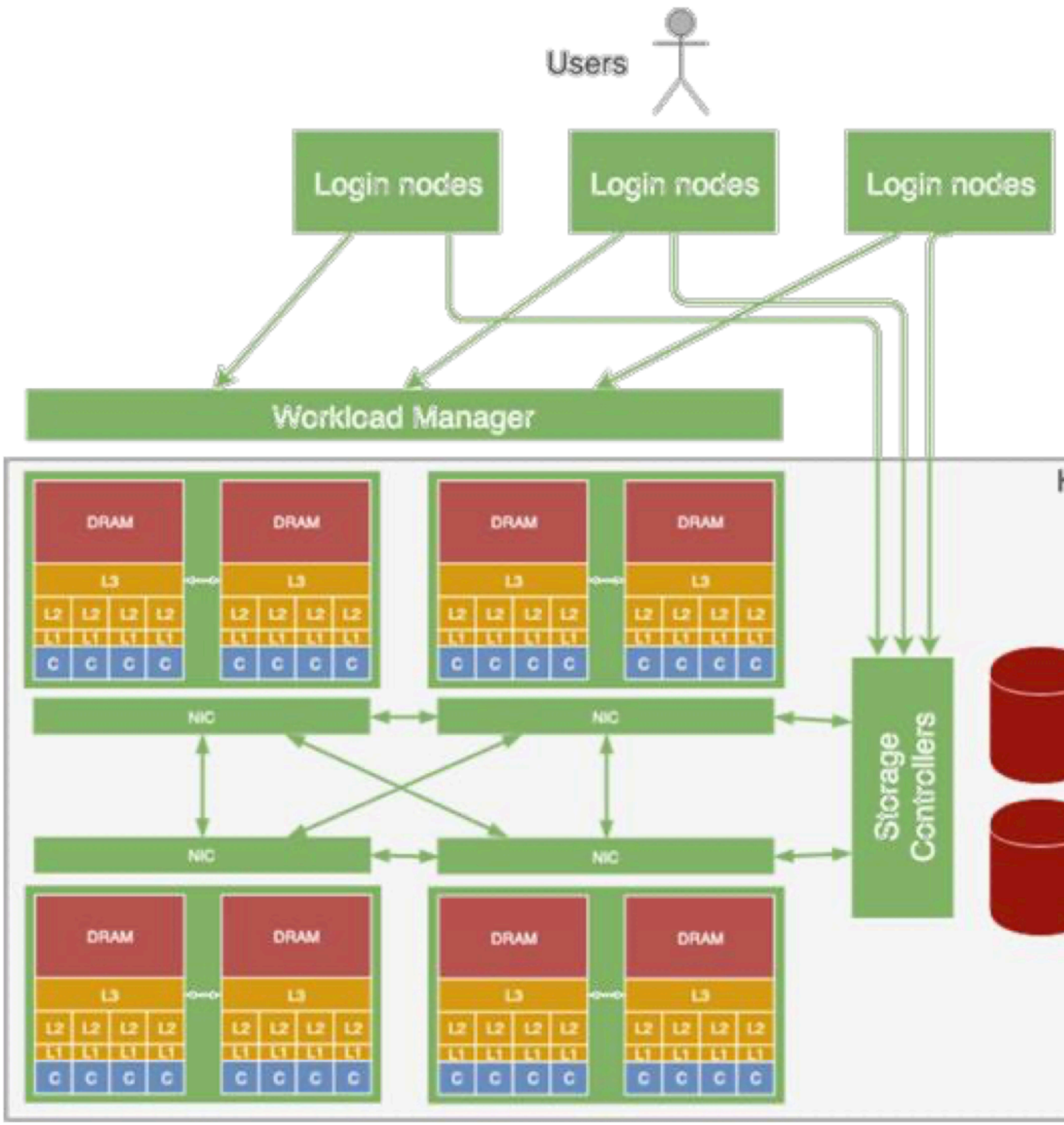
- **Moore's law**
  "*Every 18 months the power of CPU doubles.*"
- *hit the heat, size, density physics-based*
  $P \propto C * f * V^2$ *-core machine =>* ***HPC cluster***
- http://top500.org, *green500*

  (FLOP/s = Floating point operation/s)

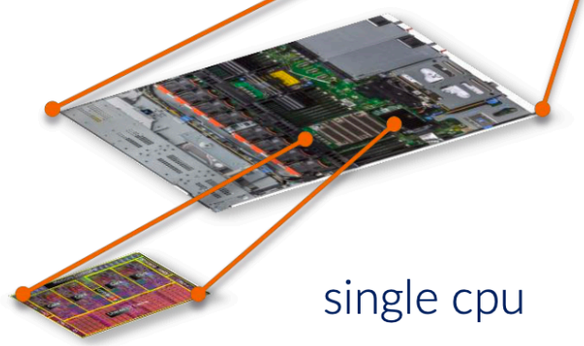
HPC includes = bunch of servers

- fast hardware
- large number of inter-connected nodes (servers), each typically multi-core (many CPUs)
- large memory
- high-speed networking
- accelerators (GPU, TPU)
- huge storage
- typically parallel algorithms
- Scheduler (Users submit **jobs** (which may include multiple tasks - parallel processes) to a scheduling queue.)
- ≠ cloud (mostly a web-service)

HPC Cluster



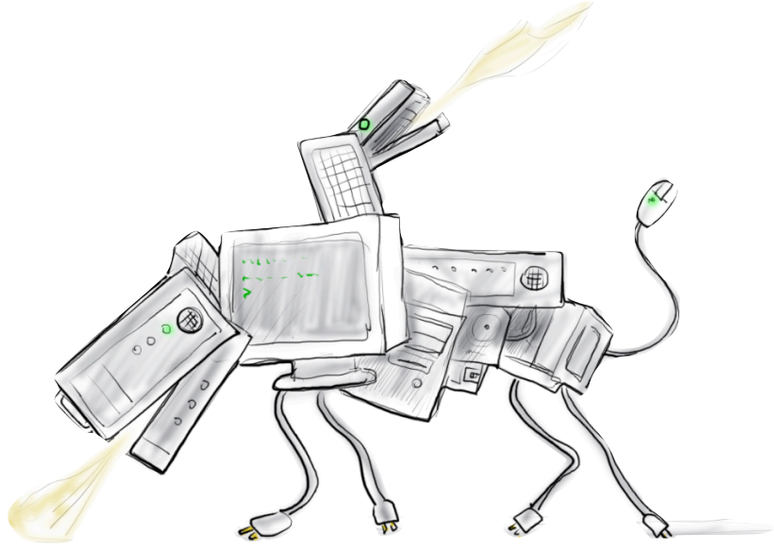
interconnected racks of connected nodes



single nodes

single cpu

Chimera



- https://hpc.cuni.cz ; https://hpc.cuni.cz/chimera/access/
- > 8838 cores on 98 **heterogeneous** computational nodes
- Air cooling 10pc, liquid cooling
- > 30 GPUs
- 500 GB DRAM
- Infiniband 200 Gb/s between some nodes
- SLURM **scheduler**
- Users submit **jobs** (which may include multiple tasks - parallel processes) to a scheduling queue.
- Admins: *Jiří Eliášek* (utf), *Ladislav Šubr* (aauk), *Zdeněk Mašín* (utf, rse)

Access to Chimera

For the 1st login use jupyterhub!

JupyterHub: https://hpc.troja.mff.cuni.cz:8000

Shutdown after use! Closing browser is not enough!

```
$> ssh <CASlogin>@hpc.troja.mff.cuni.cz
$> ssh-keygen f ~/.ssh/id_ed25519_chimera –t ed 25519
$> echo “PUBLIC_KEY” >> ~/.ssh/authorized_keys
$> chmod 700 ~/.ssh
$> chmod 600 ~/.ssh/authorized_keys

PUBLIC_KEY in ed25519_chimera.pub or
$> ssh-copy-id -i <file> <where-to>

Optional ~/.ssh/config:


Host chimera
    Hostname hpc.troja.mff.cuni.cz
    User YOUR_USERNAME
    IdentityFile ~/.ssh/id_ed25519
```

Storage

- Quotas
  — `/home` aka /troja/home 10 GB/user - mirrored (backup)
  — `/work` 100 TB, 40 GB/user
  — `/scratch` - This is a fast network-attached storage shared across all nodes (ib) — for temporary large data
  — `/scratch_local`
  — `$TMPDIR RAM` disk Gone when job finishes, counted to the memory limit of the job!

- Chimera is using LUSTRE parallel file system. The /home storage is equipped with redundancy (mirroring) to guarantee continuous access and failure protection. NFS -> `/troja/home`

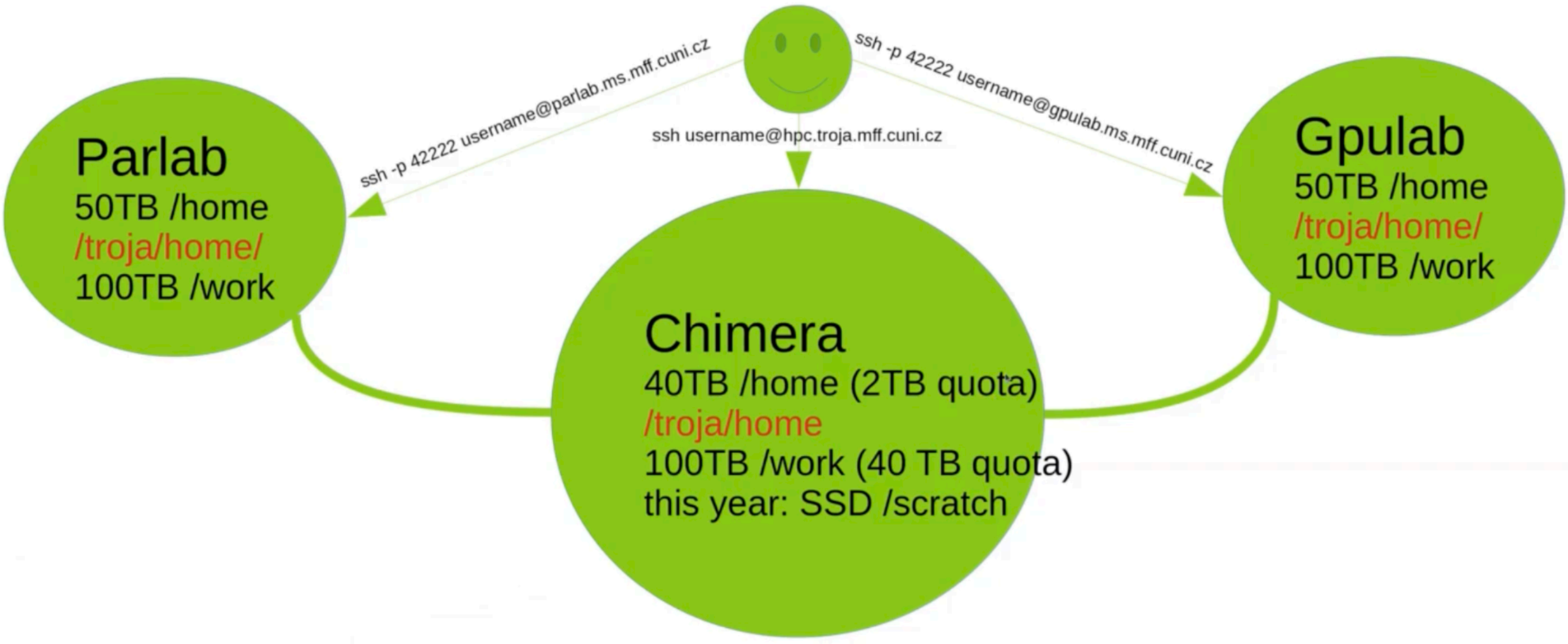
- ```
  $> lfs quota -h -u $USER /work
  $> df -h ~
  ```
  ```

File Transfer

- `scp`
- `sftp` (ls, put, get, mput, mget, lcd, pwd, lpwd, mkdir, rm, rmdir, quit, ...)
- `sshfs`
- VScode Remote-ssh-extension
- github, gitlab `$> git clone blabla.git`
- `$> rsync -avzP /path/to/file.iso <user>@hpc.troja.mff.cuni.cz:/path/to/dir/`

  `-a` archive option used to preserve file timestamps and permissions
  `-v` verbose option to monitor the transfer
  `-z` compression option to compress the file during transfer
  `-p` partial option to preserve partially transferred files in case of interruption

Chimera family




```
$> ssh -p 42222 <CASlogin>@parlab.ms.mff.cuni.cz

$> ssh -p 42222 <CASlogin>@gpulab.ms.mff.cuni.cz
```

SLURM Scheduler

Partitions (queues)

- P**artition** - a named group of compute nodes plus scheduling rules (the partitions may overlap):
  - maximum wall time
  - allowed users/accounts
  - priorities
  - available node types
  - etc.

- By default, everyone has access to partitions

  - *ffa* (free-for-all)
  - *edu*

- *Preemption (jobs can be killed if higher priority arrives)*

- Info about cluster: `lscpu`, `sview`, `sinfo`, `smypartition`, `freenodes`

- `$> sacctmgr show assoc user=$USER format=User,Account,Partition`

- `$> squeue -u $USER # same as squeue --me`

Allocating resources & Submitting jobs

- **salloc**
  `$> salloc -A account -p ffa -N 4 --ntasks-per-node 8 -t 1:00:00`

- **srun**
  `$> srun -p edu –N2 –n4 ./myprogram` or `srun p ffa –N1 -n1 --pty bash`

- **sbatch** `script.slurm`

```
#!/bin/bash
#SBATCH --job-name=my_simulation
#SBATCH --partition=ffa
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --time=01:00:00
#SBATCH --mem=8G
#SBATCH --output=logs/%x_%j.out
#SBATCH --error=logs/%x_%j.err
#SBATCH --open-mode=append
echo "Job $SLURM_JOB_NAME with ID $SLURM_JOB_ID"
echo "$SLURM_JOB_NUM_NODES Node(s): $SLURM_JOB_NODELIST"
echo "Num tasks: $SLURM_NTASKS"
echo "CPUs/task: $SLURM_CPUS_PER_TASK"
srun ./my_program
```

- Once the job gets to the front of the queue

    1) Slurm will allocate the specified number of nodes

    2) The specified script will be executed • On the first allocated node • In submit directory

    3) Once your script finishes, the job will also end

    4) stdout and stderr of your script will be written to a file on the shared filesystem slurm-$JOB_ID.out. They will be stored in the directory where you submit the job • You can override name with -o and -e

- All bash variables are exported across the nodes

- Warning: Don't forget to request GPU using `--gpus=1`, when running on gpu-xxx partitions

- `#SBATCH —exclusive`

- `srun —label ...`

Job handling

- `scancel <job-id>` — cancels a job
  `scancel -u $USER -t PENDING`

- `squeue` - view info about scheduling queue

- `sbcast` - distributes a file to the nodes allocated to a job (obsolete on Chimera)

- `sstat` - display info about a job

- `seff` - displays statistics of resources for finnished job

Modules

- `module list | avail | spider | load | unload | purge`
  `module spider GCC`

- typical use
  `$> module purge`
  `$> module load gcc`

Compile

- `$> gcc -O3 hello.c -o hello`
- Optimizations: `-O0, -O1, -O2, -O3`
- `-march=native`
  `-march=x86-64-v3`
- `-ffast-math` fast floating point, lower precision
- `-flto` link-time optimization
- `-fopenmp` enable OpenMP library
- `-lm` math library
- `-g` include debug symbols

```
#include <stdio.h>
int main(void){
    printf("Hello HPC!\n");
    return 0;
}
```

```
#!/usr/bin/env bash
#SBATCH --job-name=hello
#SBATCH --partition=edu
#SBATCH --nodes 1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATcH --time=00:05:00
#SBATCH --output=hello_%j.out

module load openapi/
intel_ipp_intel64/latest
icx hello.c O3 —march=native -o
hello
./hello
```

Optimize for the compilation host CPU. Be careful, Chimera is **heterogenous**. Choose nodes wisely. Especially when compiling on the login node.

Python *v*irtual *env*ironment

```
python3 -m venv ~/myvenv
source ~/myvenv/bin/activate
(venv) pip install -U pip setuptools wheel
(venv) pip install numpy scipy matplotlib pandas
(venv) pip install <my_package>
(venv) pip install numpy==1.26.4
deactivate
rm -rf ~/myvenv
```

Note: `python -m pip install ...`

```
python -m pip list
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
pip install -e . # developemt your own package
```

Calculate π

```python
import numpy as np
import sys

n_samples = int(sys.argv[1])
seed = int(sys.argv[2])
rng = np.random.default_rng(seed)

x = rng.random(n_samples)
y = rng.random(n_samples)
n_inside = np.count_nonzero(x*x + y*y <= 1)

result = 4*n_inside/n_samples
print(result)
```

Parallel π #1

```
#!/bin/bash
N=100000
for seed in {0..3}; do
  sbatch pi_job.slurm "$N" "$seed"
done
```

```
#!/bin/bash
#SBATCH --job-name=calc_pi
#SBATCH --partition=ffa
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --time=00:05:00
#SBATCH --output=pi_%j.out
python calc_pi.py "$1" "$2"
```

Parellel π #2

```
#!/bin/bash

#SBATCH --job-name=calc_pi
#SBATCH --partition=edu
#SBATCH --nodes=1
#SBATCH --ntasks=4
##SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --time=00:05:00
#SBATCH --output=pi_%j.out

# Run 8 independent calculations, each with a unique seed
srun python calc_pi.py 100000 "$SLURM_PROCID"

# All tasks have finished. Calculate the average.
awk '/^[0-9]+\.[0-9]+$/ {sum += $1; n++}
     END {if (n > 0) printf "Average pi = %.10f (%d tasks)\n",
sum/n, n}' \
    "pi_${SLURM_JOB_ID}.out"
```

Parellel π #2 (correct)

```
#!/bin/bash

#SBATCH --job-name=calc_pi
#SBATCH --partition=edu
#SBATCH --nodes=2
#SBATCH --ntasks=8
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --time=00:05:00
#SBATCH --output=pi_%j.out

# Run 8 independent calculations, each with a unique seed
srun bash -c 'python calc_pi.py 100000 "$SLURM_PROCID"'

# All tasks have finished. Calculate the average.
awk '/^[0-9]+\.[0-9]+$/ {sum += $1; n++}
     END {if (n > 0) printf "Average pi = %.10f (%d tasks)\n",
sum/n, n}' \
    "pi_${SLURM_JOB_ID}.out"
```

Job-array *hello*

```
#!/bin/bash
#SBATCH --partition ffa
#SBATCH --job-name "test_job_array"
#SBATCH --cpus-per-task 1
#SBATCH --mem-per-cpu 100m
#SBATCH --time 0:10:00
#SBATCH --array 1-32%8 # duplicate the job 32 times - slurm
sets SLURM_ARRAY_TASK_ID environment variable which
corresponds to the index each job in the array
#SBATCH --error=error_%a.txt              # redirects stderr to
error_${SLURM_ARRAY_TASK_ID}.txt
#SBATCH --output=output_%a.txt            # redirects stdout to
output_${SLURM_ARRAY_TASK_ID}.txt

name=$(hostname)
echo "Hello from $name (job: $SLURM_ARRAY_JOB_ID) I am a sub-
job  $SLURM_ARRAY_TASK_ID"

sleep 300s
```

Parallel π #3 — job-array version

```
#!/bin/bash
#SBATCH --job-name=calc_pi
#SBATCH --partition=edu
#SBATCH —array=0-7 # %2
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --time=00:05:00
#SBATCH --output=pi_%A_%a.out

python calc_pi.py 100000 "$SLURM_ARRAY_TASK_ID"
```

Running on other cluster

```
#!/bin/bash
#SBATCH --clusters parlab # on other sub-cluster
#SBATCH --partition ffa
#SBATCH --job-name test
#SBATCH --cpus-per-task 8
#SBATCH --mem-per-cpu 2g
#SBATCH --time 0:10:00

name=$(hostname)
echo "Hello from" $name
cat /troja/home/topim5am/git/clusters/README.md #
be sure it is /troja/home, not /home
sleep 30s
squeue -M parlab
```

Memory estimate

```
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=16G
#SBATCH --mem-per-cpu=4G
```

```
import numpy as np
data = np.zeros((1000, 1000, 100), dtype=np.float64)
print(data.nbytes / 1024**3, "GiB")

$> /usr/bin/time

User time (seconds): 125.34
System time (seconds): 4.12
Percent of CPU this job got: 395%
Elapsed (wall clock) time: 0:32.81
Maximum resident set size (kbytes): 12582912
```

Rule of thumb: Set peak performance x 1.3

Checkpointing

```
#!/bin/bash
#SBATCH --job-name=simulation
#SBATCH --partition=ffa-preempt
#SBATCH --time=02:00:00
#SBATCH --signal=B:USR1@120

module load DMTCP
source /usr/local/bin/scheckpoint \
    --tiger-jump \
    --tiger-jump-size 00:05:00 \
    python simulation.py
```

- `--tiger-jump`, if used, the simulation is started on the '*checkpoint*' partition on an old hardware to ensure instruction compatibility with the rest of the nodes. It is useful for some compiled binaries. `scheckpoint` uses DMTCP under the hood.
  Note the use of source, to ensure it does not create a new process and therefore can listen to SLURM signals.
- Warning: Does not work across multiple nodes, such as in MPI.
- Warning: Does not work with GPU.

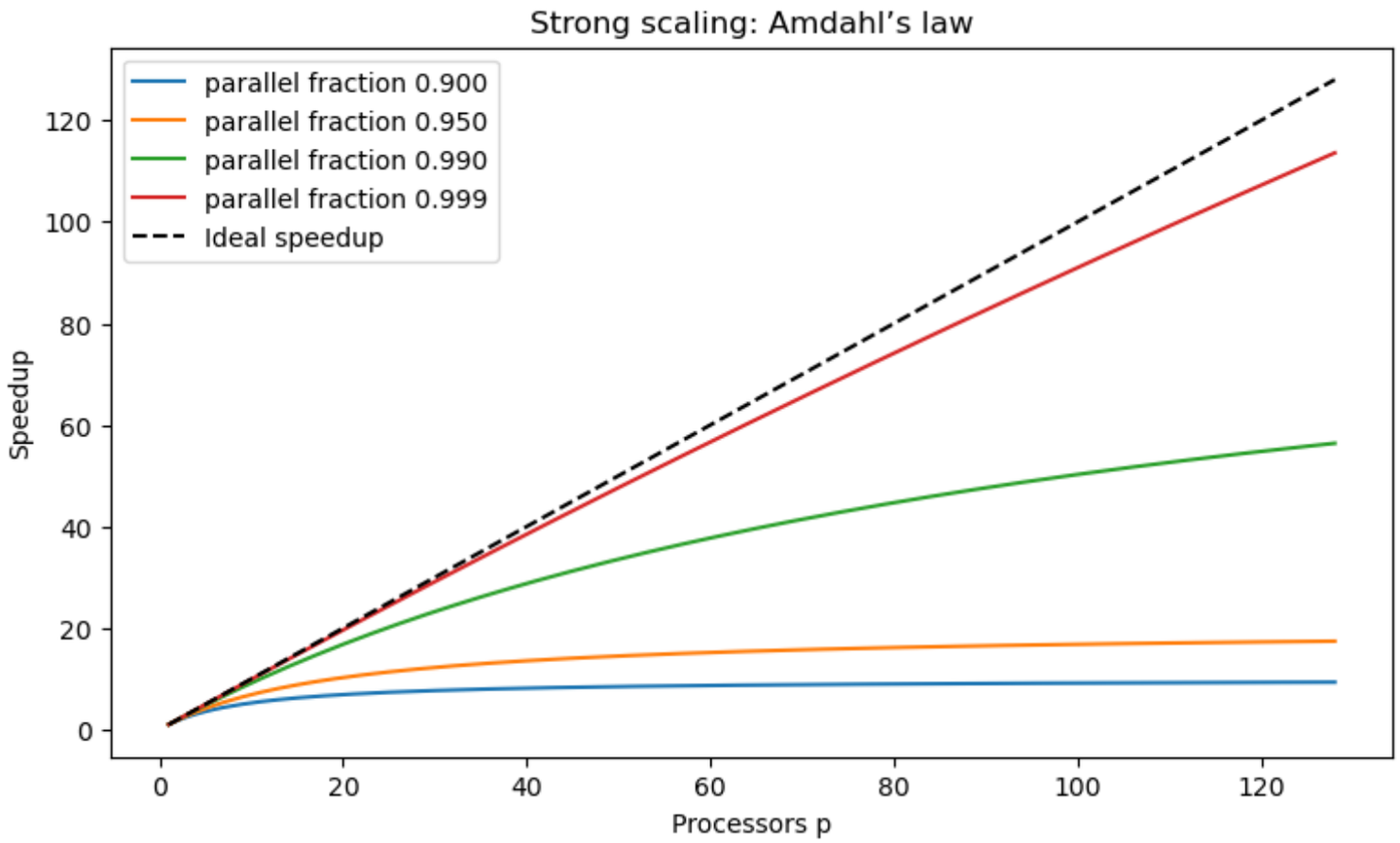
Parallel Programming

Amdahl's law

- If *f* is the fully parallel fraction, serial part time $T_s = (1-f)T_1$, parallel part $T_p = f\,T_1/p \Rightarrow T = T_1\,[\,(1-f) + f/p]$
- Defining speed-up $S(p) = T_1/T_p$

$$S(p) = \frac{1}{(1-f) + f/p}$$

- Ignoring overheads, initialisation, communication, I/O, memory transfers, synchronisation



f = 0.9, p = 1000 cores:
speed-up = 9.91
efficiency [S/p] = 1.0%
S_{max} = 10

- **strong scaling** -- the same amount of data faster
- **weak scaling** -- bigger data in same amount of time

Going parallel

Data decomposition — embarrassing parallelism

Domain decomposition

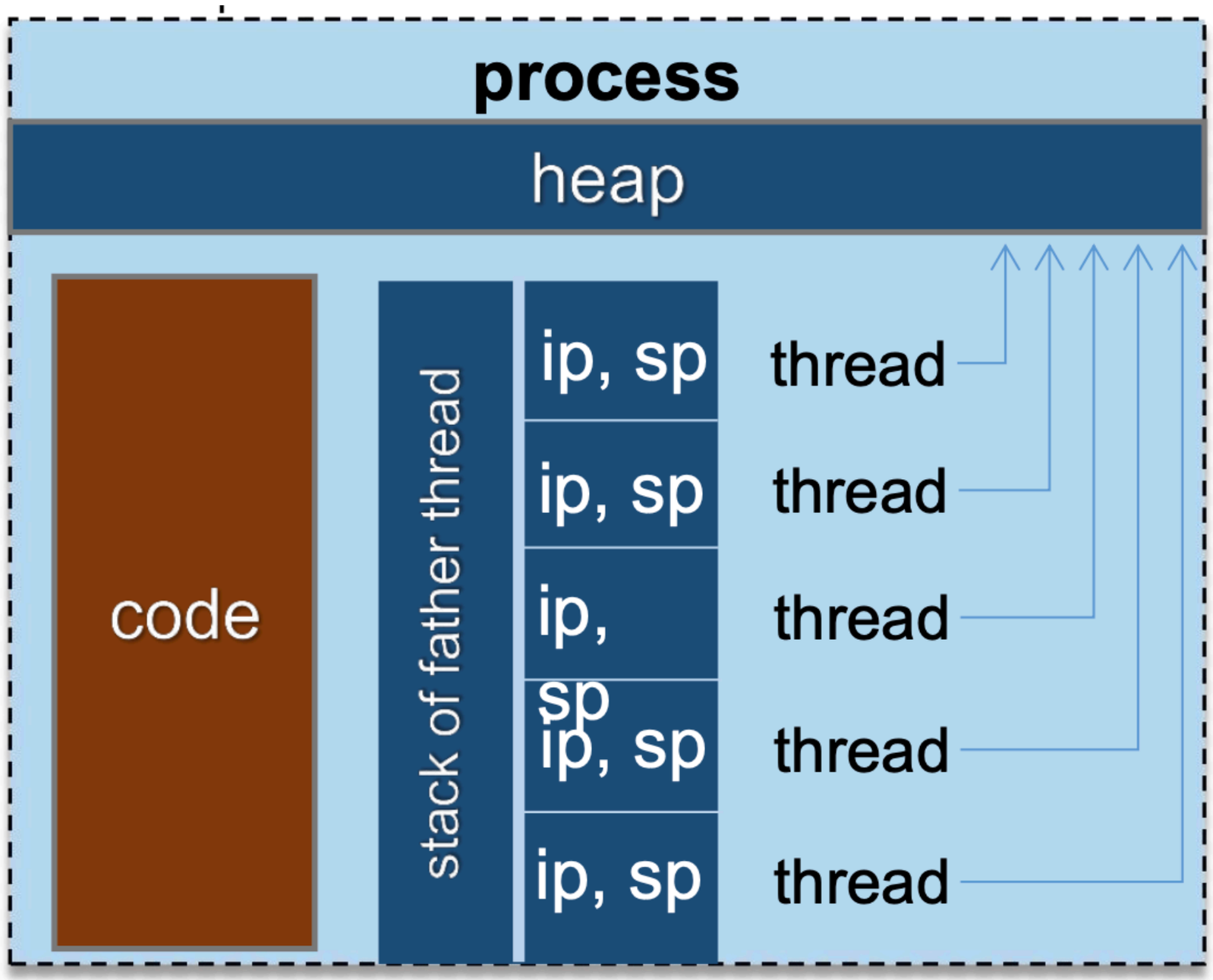
Flynn's Taxonomy:

	Single instruction	**Multiple instructions**
Single data	SISD (old scalar CPU)	MISD (rare, fault-tolerant systems)
Multiple data	SIMD (vector units, GPUs)	MIMD (multicore CPUs, clusters)

Threads vs Processes

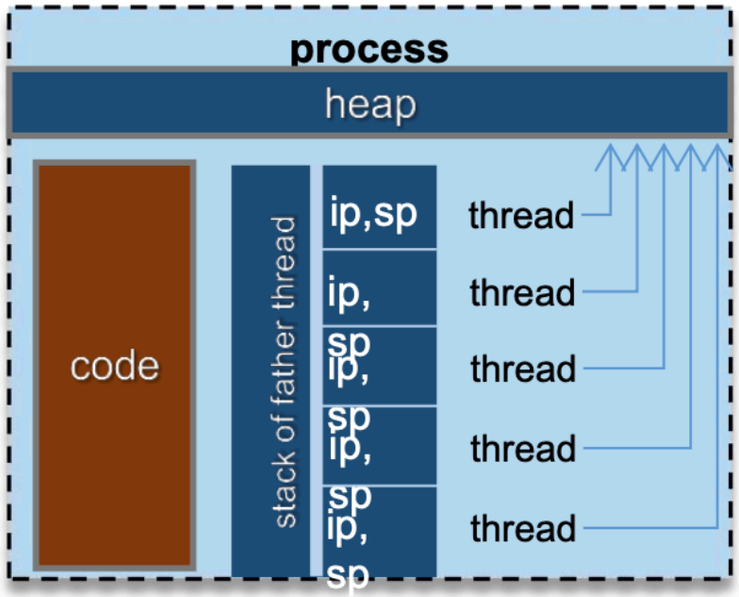
?

Threads vs Processes



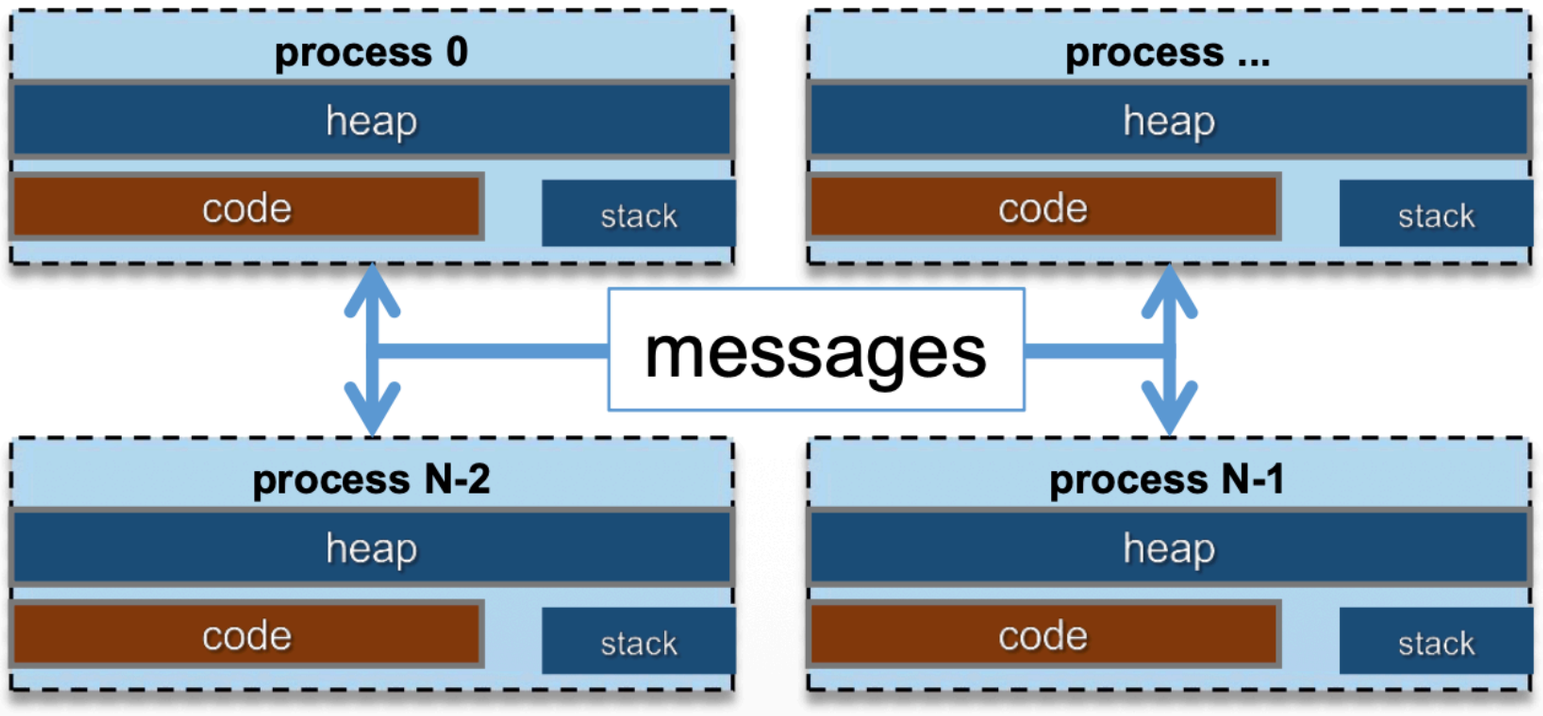
Shared-Memory
(e.g. OpenMP)

A unique process that spawns a number of threads. There is a unique memory space that is accessible by all the threads



Distributed-Memory
(e.g. MPI)

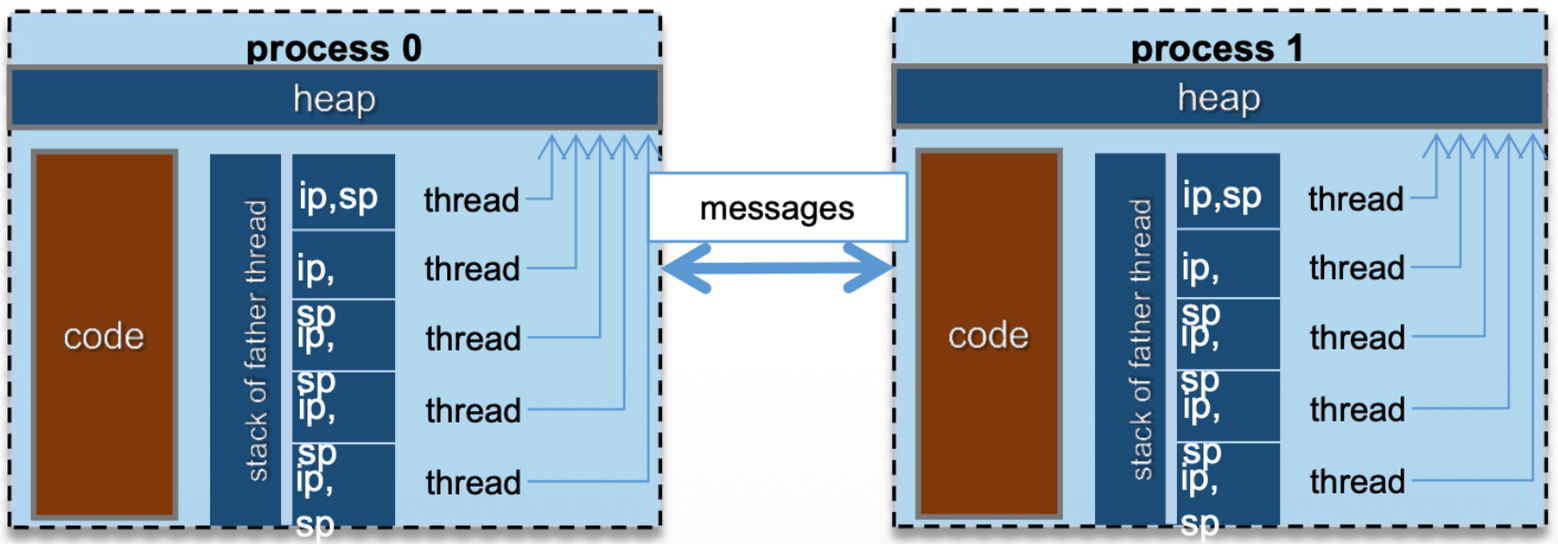
N processes are created, each with its own copy of the code and its own memory space.
A process ***can not*** access the memory space of another process.
The processes communicate through *messages*.



- **Does SLURM create the threads? No!**
  SluRM allocates CPU resources and runs tasks. It is the application that creates its own software threads and they run on allocated cores (CPUs).

Distributed-Memory
(e.g. MPI) **+ Shared-Memory**

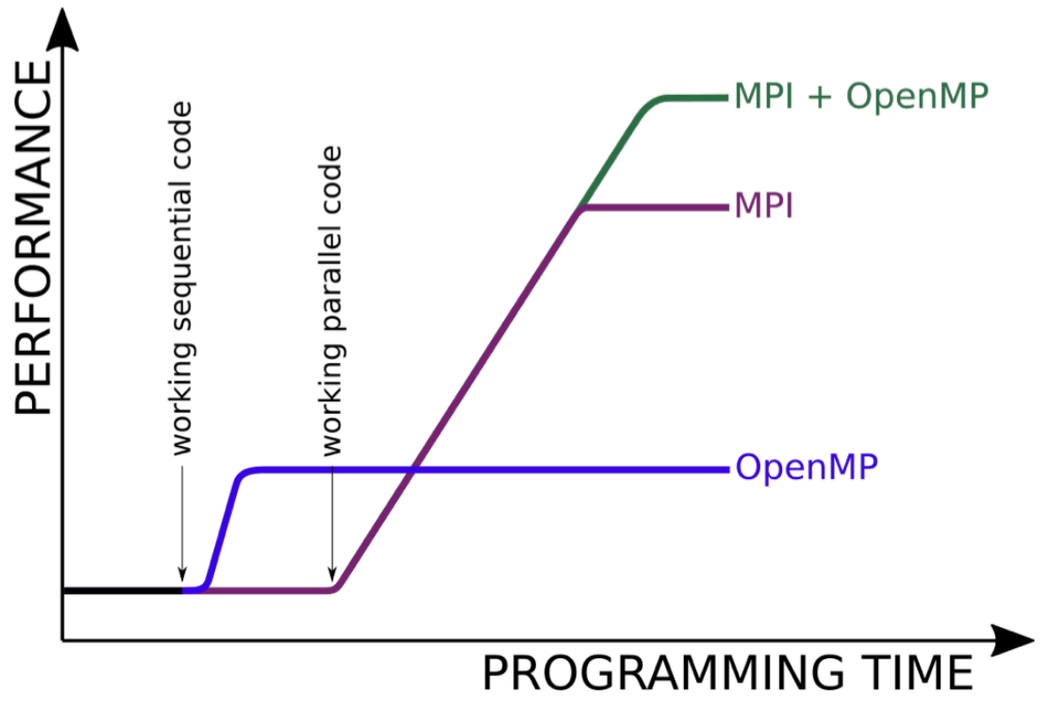
N processes are created, each with its own copy of the code and its own memory space. Each process may spawn a number of threads as in shared-memory.
A process *can not* access the memory space of another process (nor any of its threads can). The processes communicate through *messages*.



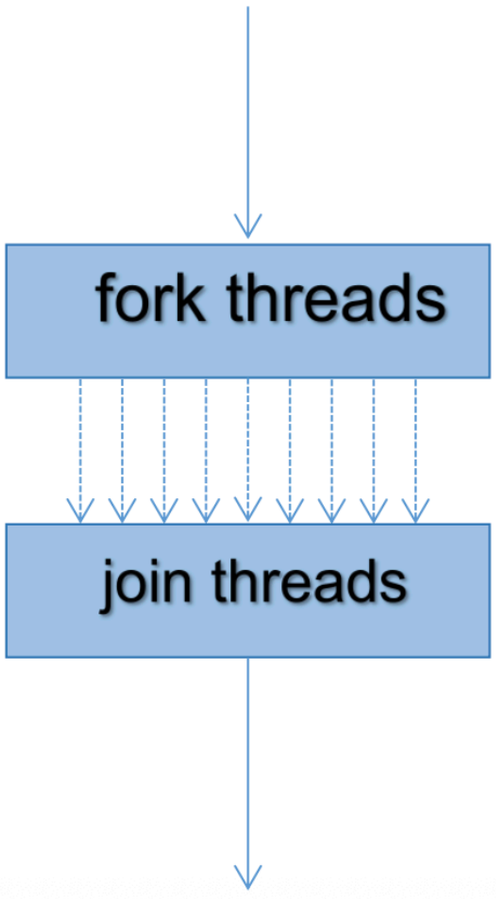
- **shared memory**
  -- threads (openMP)

- **distributed memory**,
  communication -- processes (MPI)

- **hybrid**, processes with treads
  -- (mixture of MPI w/openMP)



openMP (threads w/shared memory)



Master thread

the master thread spawns children and joins the new pool as thread 0

Team of threads

Master thread

```
#include <stdio.h>
#include <omp.h>

int main(void) {
  #pragma omp parallel
  {
    int tid = omp_get_thread_num();
    int nthreads =
omp_get_num_threads();
    printf("Hello from thread %d of
%d\n", tid, nthreads);
  }
  return 0;
}
```

```
$> gcc -fopenmp hello.c -o hello
$> OMP_NUM_THREADS=4 ./hello
```

For Fortran lovers

- C/C++:

```
#include <omp.h>
#pragma omp paralel
{
omp_get_thread_num();
omp_get_num_threads();
…
}
```

- Fortran:

```
USE OMP_LIB
!OMP PARALLEL
…
OMP_GET_THREAD_NUM()
OMP_GET_NUM_THREADS(
)
!OMP END PARALLEL
```

Variable scopes

```
#include <stdio.h>
#include <omp.h>

int main(void) {
  int a = 1;
  int b = 3;
  int c = 10;
  double value;

  #pragma omp parallel firstprivate(c)
  {
    int i = 1;
    int result;
    b = 10*b
    result = a + b + c;
    value = result;
  }

  return 0;
}
```

shared(x)

— lives on heap or the stack of the father

— before the parallel region

private(x)

— lives in the stack of the thread

— inside the parallel region

firstprivate(x)

— like private with copied value

parallel for

```
#pragma omp parallel
{
  int tid = omp_get_thread_num();
  int nthreads = omp_get_num_threads();

  int start = tid * N / nthreads;
  int end = (tid + 1) * N / nthreads;
  for (int i = start; i < end; i++) {
     … bla bla bla bla calculation …
  }
```

Divide iterations equally among threads

```
#pragma omp parallel
{
#pragma omp for
{
  for (int i = start; i < end; i++)
{
     … bla bla bla bla calculation …
  }
}
```

Options:

`schedule(static|dynamic)`

`collapse(n)`

One-liner:

`#pragma omp parallel for`

Back to π

$$\pi = \int_0^1 \frac{4}{1+x^2}\,dx$$

Using the **midpoint rule** with N intervals:

$$\pi \approx \frac{1}{N}\sum_{i=0}^{N-1} \frac{4}{1+\left(\frac{i+0.5}{N}\right)^2}$$

```
#include <stdio.h>
#include <omp.h>
#define N 100000
int main(void) {
    double sum = 0.0;
    double dx = 1.0 / N;
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        double x = (i + 0.5) * dx;
        sum += 4.0 / (1.0 + x*x);
    }
    double pi = sum * dx;
   printf("Pi = %.15f\n", pi);
   return 0;
}
```

Data race condition

```
#include <stdio.h>
#include <omp.h>
#define N 100000
int main(void) {
    double sum = 0.0;
    double dx = 1.0 / N;
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        double x = (i + 0.5) * dx;
        sum += 4.0 / (1.0 + x*x);
    }
    double pi = sum * dx;
   printf("Pi = %.15f\n", pi);
   return 0;
}
```

Data race →

Barriers

- `#pragma omp barrier`
- `#pragma omp atomic`
  `assignment`
- `#pragma omp critical`
  `{`
  `…`
  `}`

Data race condition — fixed

```
#include <stdio.h>
#include <omp.h>
#define N 100000
int main(void) {
    double sum = 0.0;
    double dx = 1.0 / N;
    #pragma omp parallel
    {
     double local_sum = 0;
     #pragma omp for
     for (int i = 0; i < N; i++) {
       double x = (i + 0.5) * dx;
       local_sum += 4.0 / (1.0 + x*x);
       }
     #pragma omp atomic
     sum += local_sum
     }
   double pi = sum * dx;
   printf("Pi = %.15f\n", pi);
   return 0;
}
```

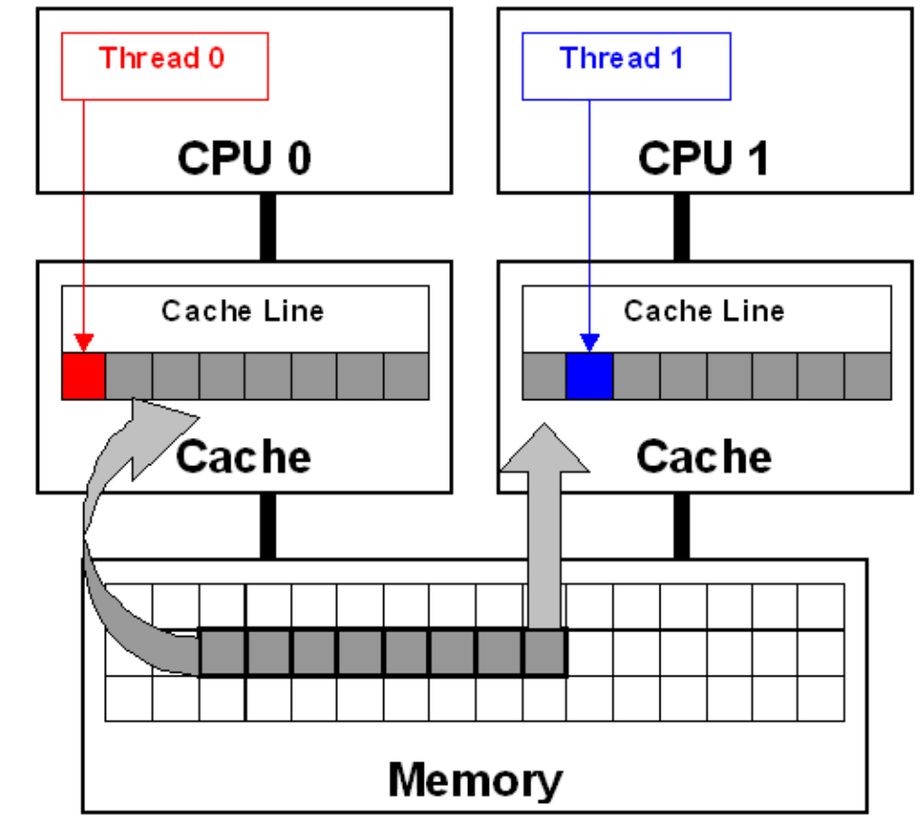
reduction (op:var)

```
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < N; i++) {
double x = (i + 0.5) * dx;
sum += 4.0 / (1.0 + x*x);
}
```

Where op is +, -, min, max, *, …

False Sharing

```
#include <omp.h>

int main(void) {
   double results[8];
   #pragma omp parallel num_threads(8)
   {
      int tid = omp_get_thread_num();
      results[tid] = calculation(…, tid, …)
   }
   return 0;
}
```



Python & Threads

- Warning: Python has GIL (Global Interpreter Lock) — only 1 thread a time!
- However:

  — C libraries (some numpy, scipy, tf, pytorch, pandas, BLAS, LAPACK…) pass over

  — JIT: **numba**, Cython

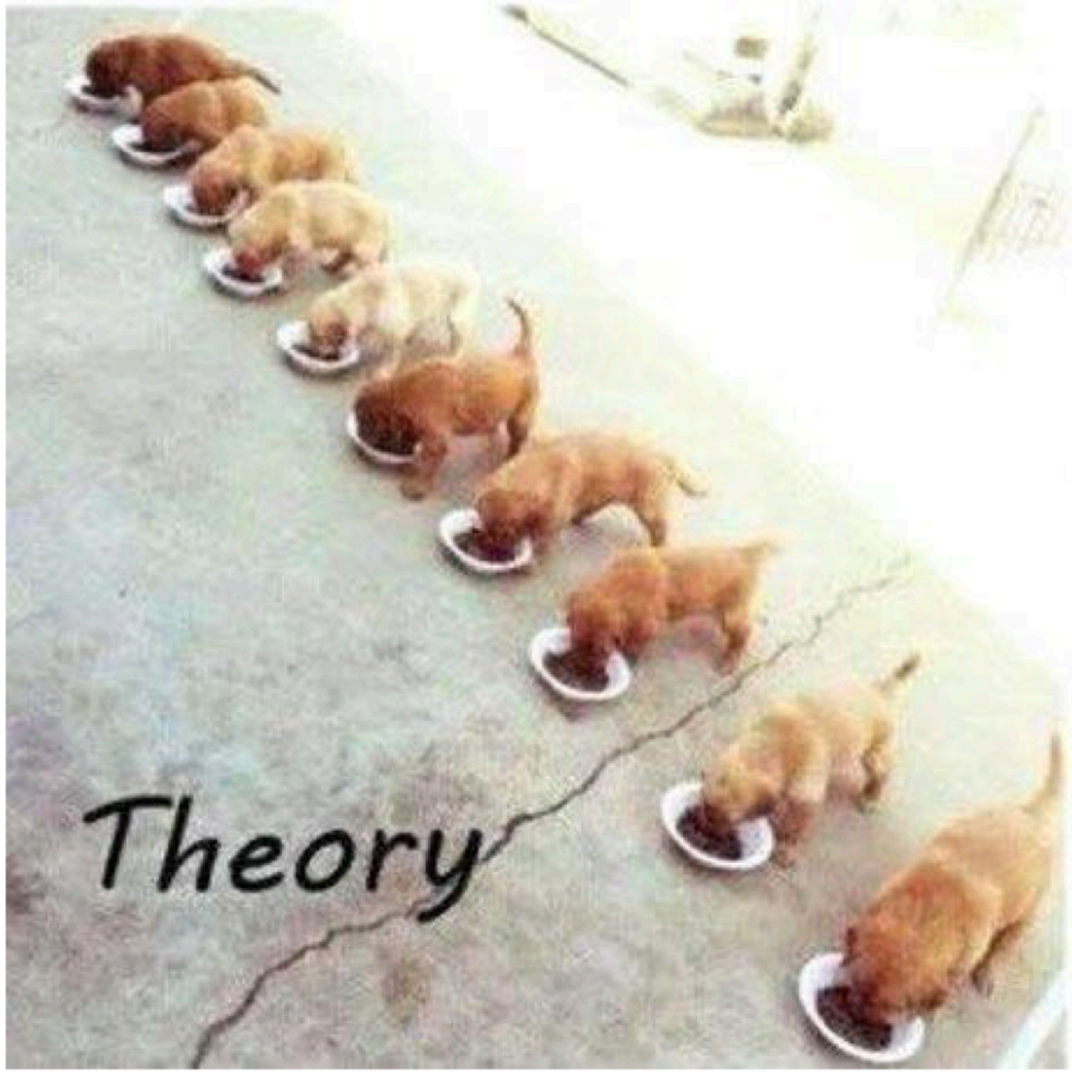
Numba rulez!

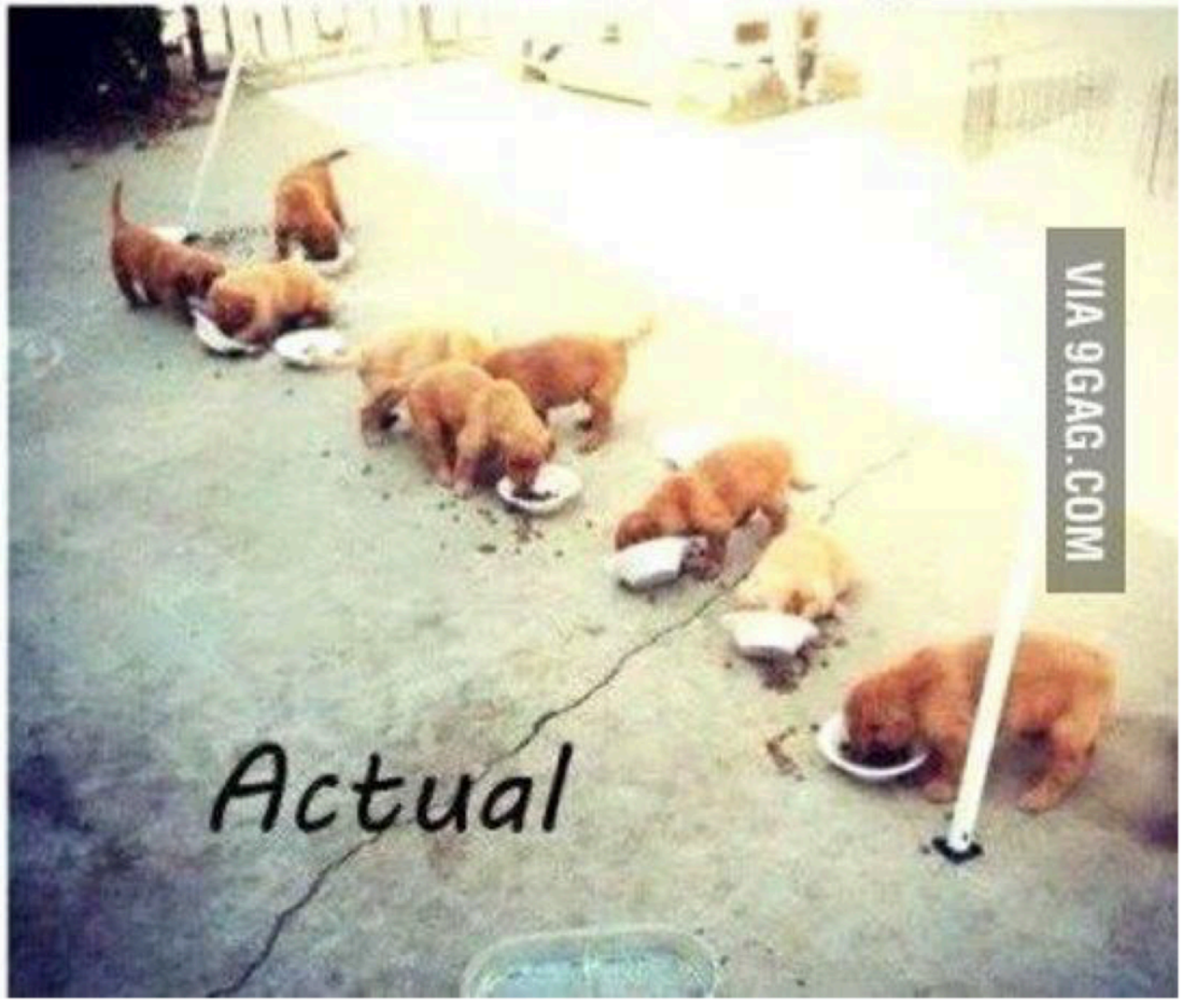
```
from numba import njit, prange

@njit(parallel=True, nogil=True)
def calc_pi(N):
    dx = 1.0 / N
    total = 0.0

    for i in prange(N):
        x = (i + 0.5) * dx
        total += 4.0 / (1.0 + x*x)
    return total * dx
```

openMP summary

- Super easy -> `#pragma omp`
- Super powerful (we just scratched the surface…, we did not mention *sections, tasks, places, binding*…)
- Limit is #cores and memory bandwidth
- `export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}`

Multithreaded programming

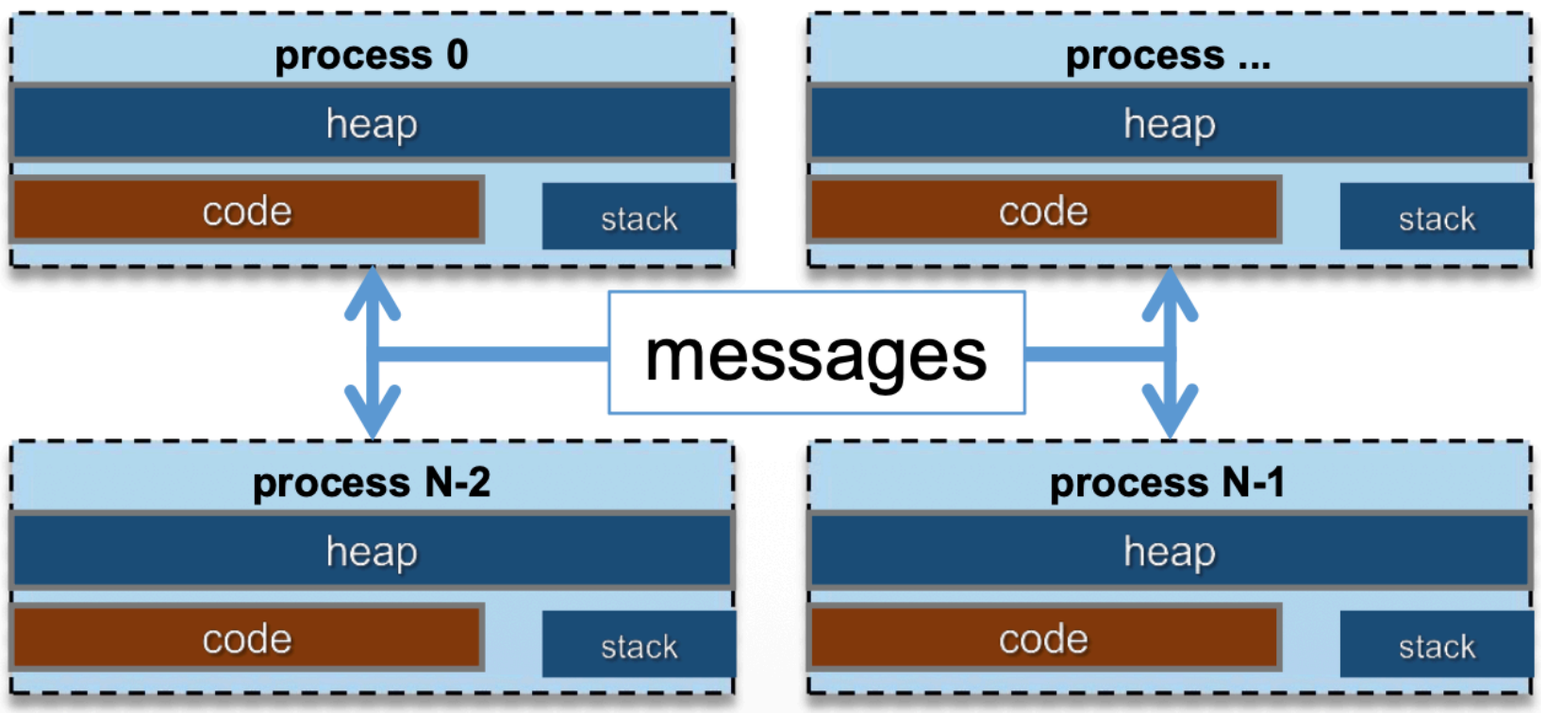




Message Parsing Interface (MPI)

Distributed-Memory
(e.g. MPI)

N processes are created, each with its own copy of the code and its own memory space.
A process ***can not*** access the memory space of another process.
The processes communicate through *messages*.



- Each MPI process has its own memory space.
- Processes communicate explicitly by sending and receiving messages.
- Unlike OpenMP, MPI does not require shared memory.
- MPI works within a single node and across multiple nodes.
- Most MPI programs follow the SPMD model: Single Program, Multiple Data

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Hello from rank %d of %d\n", rank, size);

    MPI_Finalize();
    return 0;
}
```

MPI library

Setup layout
and communicator

rank (process id)

size

Final synchronisation

Run MPI

- Intel MPI: Load via `module load oneapi/mpi`. Use `mpiicc`
- `openmpi-5.0.8/ib`: Default for *InfiniBand* nodes, request SLURM resources with `-C ib`, otherwise `openmpi-5.0.8/tcp`

```
$> mpicc hello.c -o hello
$> mpirun -np 4 ./hello
```

On a SLURM cluster:

```
mpirun --mpi=pmix -n 4 ./hello
srun --mpi=pmix -n 4 ./hello
~~srun mpirun --mpi=pmix -n 4 ./hello~~
```

Calculate π

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    const long N = 1000000;
    double dx = 1.0 / N;
    double local_pi = 0.0;

    // each process calculates its part
    for (long i = rank; i < N; i += size)
{
        double x = (i + 0.5) * dx;
        local_pi += 4.0 / (1.0 + x*x) *
dx;
    }

    if (rank == 0) {
        // Master collects and sums
results
        double pi = local_pi;

        for (int i = 1; i < size; i++) {
            double received;
            MPI_Recv(&received, 1,
MPI_DOUBLE, i, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
            pi += received;
        }
        printf("Pi = %.12f\n", pi);
    } else {
        // Workers send results to master
        MPI_Send(&local_pi, 1, MPI_DOUBLE,
                 0, 0, MPI_COMM_WORLD);
    }
    MPI_Finalize();
    return 0;
}
```

Map Reduce

```
double pi = 0.0;

MPI_Reduce(&local_pi, &pi, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);


if (rank == 0)
    printf("Pi = %.12f\n", pi);
```

Function	C syntax	Description
MPI_Send	`MPI_Send(buf, count, datatype, dest, tag, comm)`	Blocking send to another process
MPI_Recv	`MPI_Recv(buf, count, datatype, source, tag, comm, status)`	Blocking receive from another process
MPI_Reduce	`MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, comm)`	Combine values from all processes; result on root
MPI_Allreduce	`MPI_Allreduce(sendbuf, recvbuf, count, datatype, op, comm)`	Combine values; result available to all processes
MPI_Gather	`MPI_Gather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm)`	Collect equal-sized contributions from all processes onto root
MPI_Allgather	`MPI_Allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm)`	Gather data and distribute the complete result to all processes
MPI_Gatherv	`MPI_Gatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs, recvtype, root, comm)`	Gather variable-sized contributions onto root

MPI_Sendrecv	`MPI_Sendrecv(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount, recvtype, source, recvtag, comm, status)`	Send and receive in one operation
MPI_Isend	`MPI_Isend(buf, count, datatype, dest, tag, comm, request)`	Non-blocking send
MPI_Irecv	`MPI_Irecv(buf, count, datatype, source, tag, comm, request)`	Non-blocking receive
MPI_Barrier	`MPI_Barrier(comm)`	Wait until all processes reach the barrier
MPI_Bcast	`MPI_Bcast(buffer, count, datatype, root, comm)`	Broadcast data from root to all processes
MPI_Wait	`MPI_Wait(request, status)`	Wait for one non-blocking operation to complete
MPI_Waitall	`MPI_Waitall(count, requests, statuses)`	Wait for multiple non-blocking operations to complete

MPI in Python

```
from mpi4py import MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

print(f"Hello from rank {rank} of {size}”)

_ _ _
srun -n 4 python hello.py
```

Calculate π — MPI in Python

```
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

N = 100_000_000
dx = 1.0 / N

# MAP: each process calculates
its own samples
i = np.arange(rank, N, size)
x = (i + 0.5) * dx
local_pi = np.sum(4.0/
(1.0+x*x))*dx
```

```
if rank == 0:
    # Master collects and sums
    pi = local_pi
    for source in range(1, size):
        received =
comm.recv(source=source, tag=0)
        pi += received
    print(f"Pi = {pi:.12f}")

else:
    # Workers send partial
results
    comm.send(local_pi, dest=0,
tag=0)
```

Deadlock communication

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    int send = rank;
    int recv;
    int other = 1 - rank;
    MPI_Send(&send, 1, MPI_INT, other, 0, MPI_COMM_WORLD);
    MPI_Recv(&recv, 1, MPI_INT, other, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Rank %d received %d\n", rank, recv);
    MPI_Finalize();
    return 0;
}
```

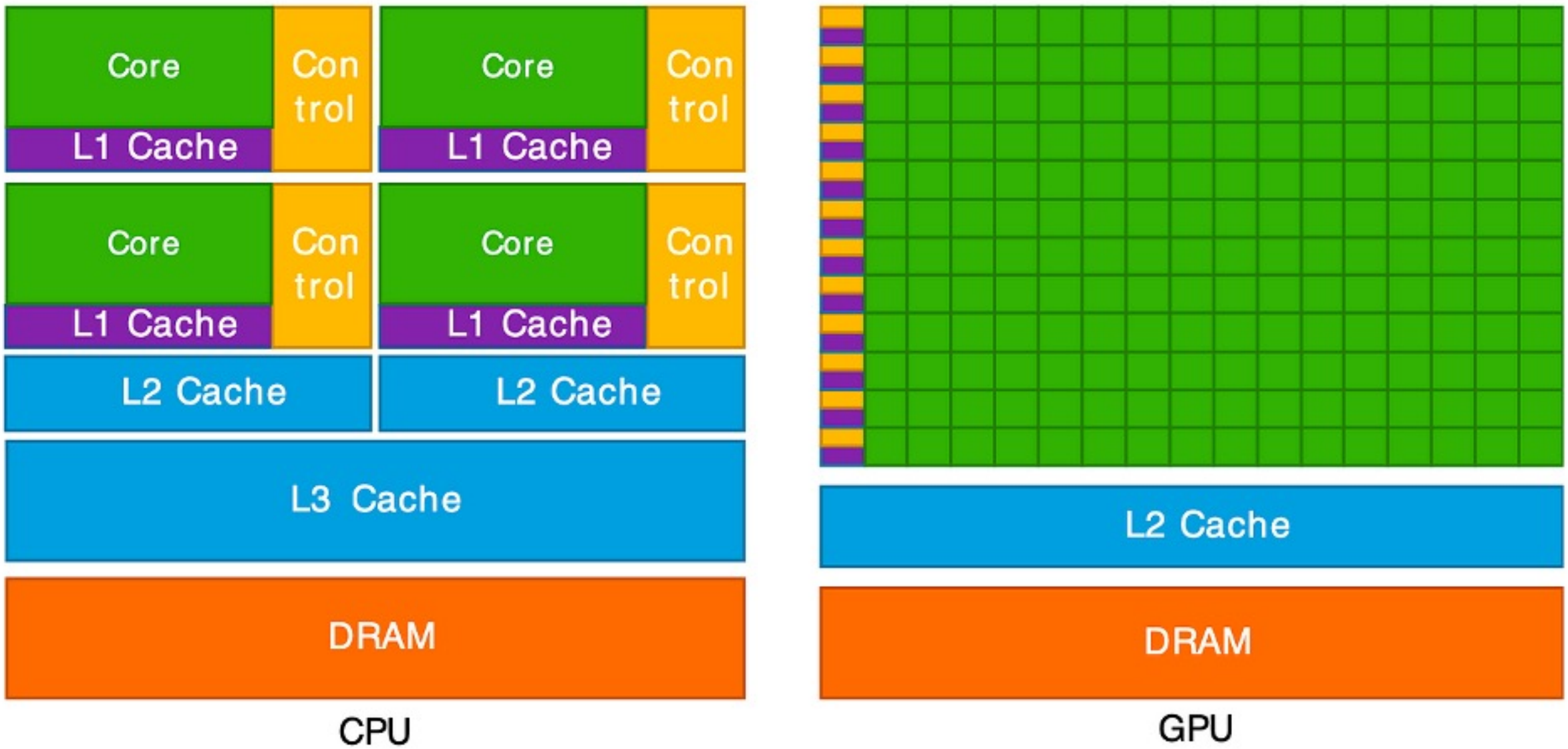
Non-blocking communication

- ```
  if (rank == 0) {
  MPI_Send(&send, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
  MPI_Recv(&recv, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
  MPI_STATUS_IGNORE);
  } else {
  MPI_Recv(&recv, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
  MPI_STATUS_IGNORE);
  MPI_Send(&send, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);}
  ```

- **MPI_Sendrecv**(&send, 1, MPI_INT, other, 0, &recv, 1, MPI_INT, other, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

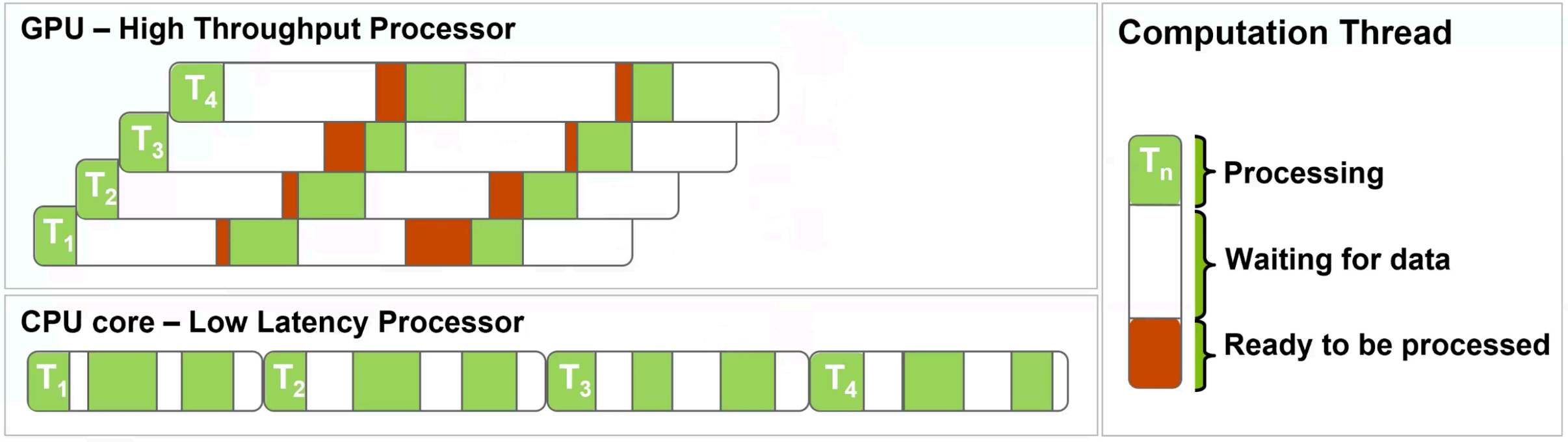
- ```
  MPI_Request req[2];
  ```
  MPI_Irecv(&recv, 1, MPI_INT, other, 0, MPI_COMM_WORLD, &req[0]);
  MPI_Isend(&send, 1, MPI_INT, other, 0, MPI_COMM_WORLD, &req[1]);
  MPI_Waitall(2, req, MPI_STATUSES_IGNORE);

GPU — massively parallel



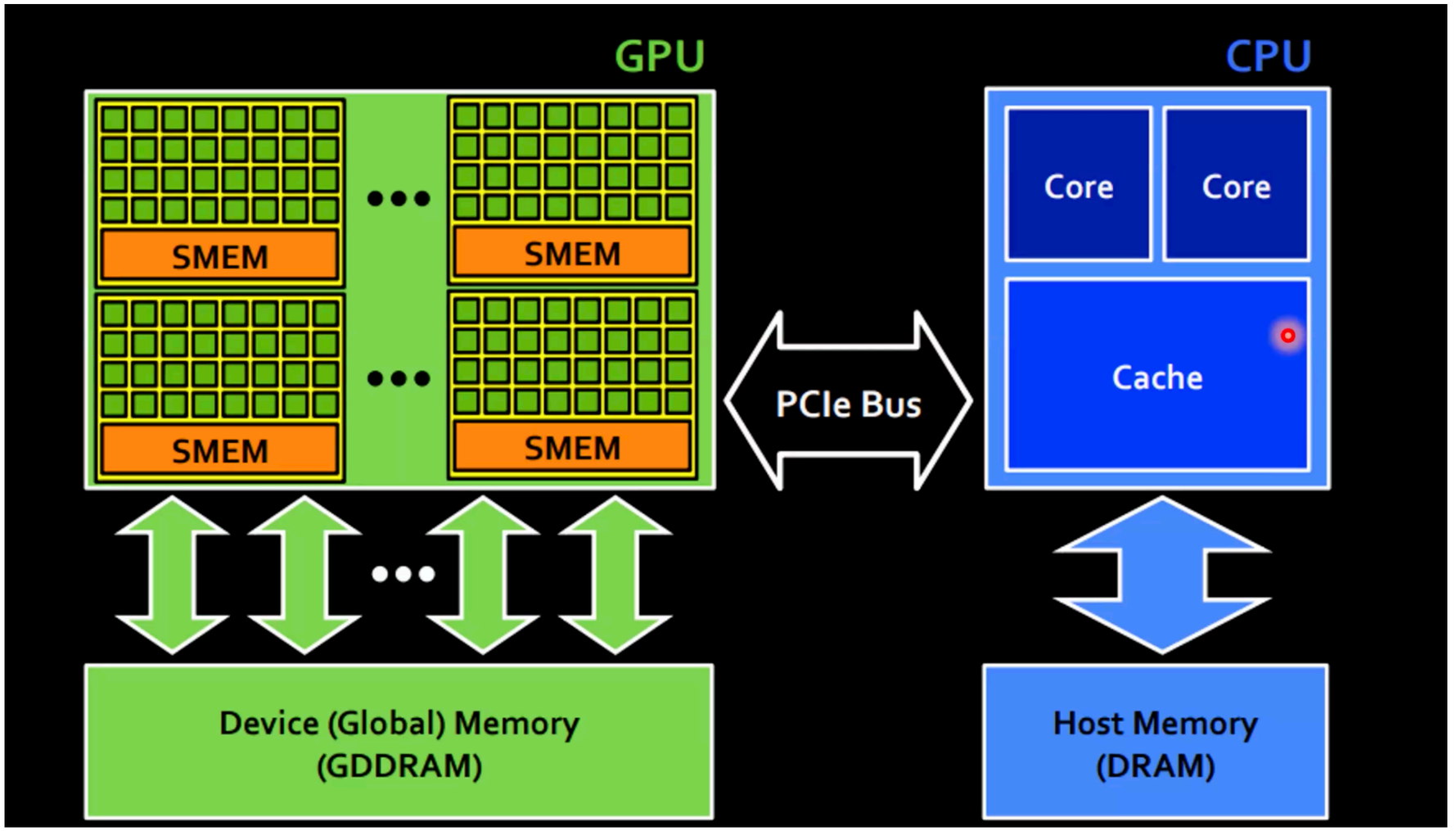
off-load work from ***host*** to ***device*** and back

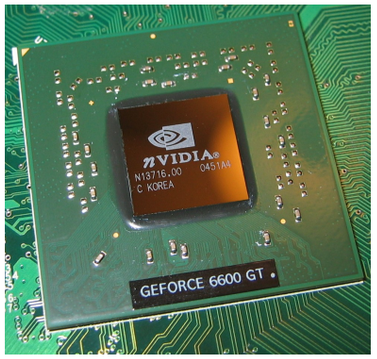
- CPU architecture must minimize latency within each thread
- GPU architecture hides latency with computation (data-parallelism, to 30k threads!)



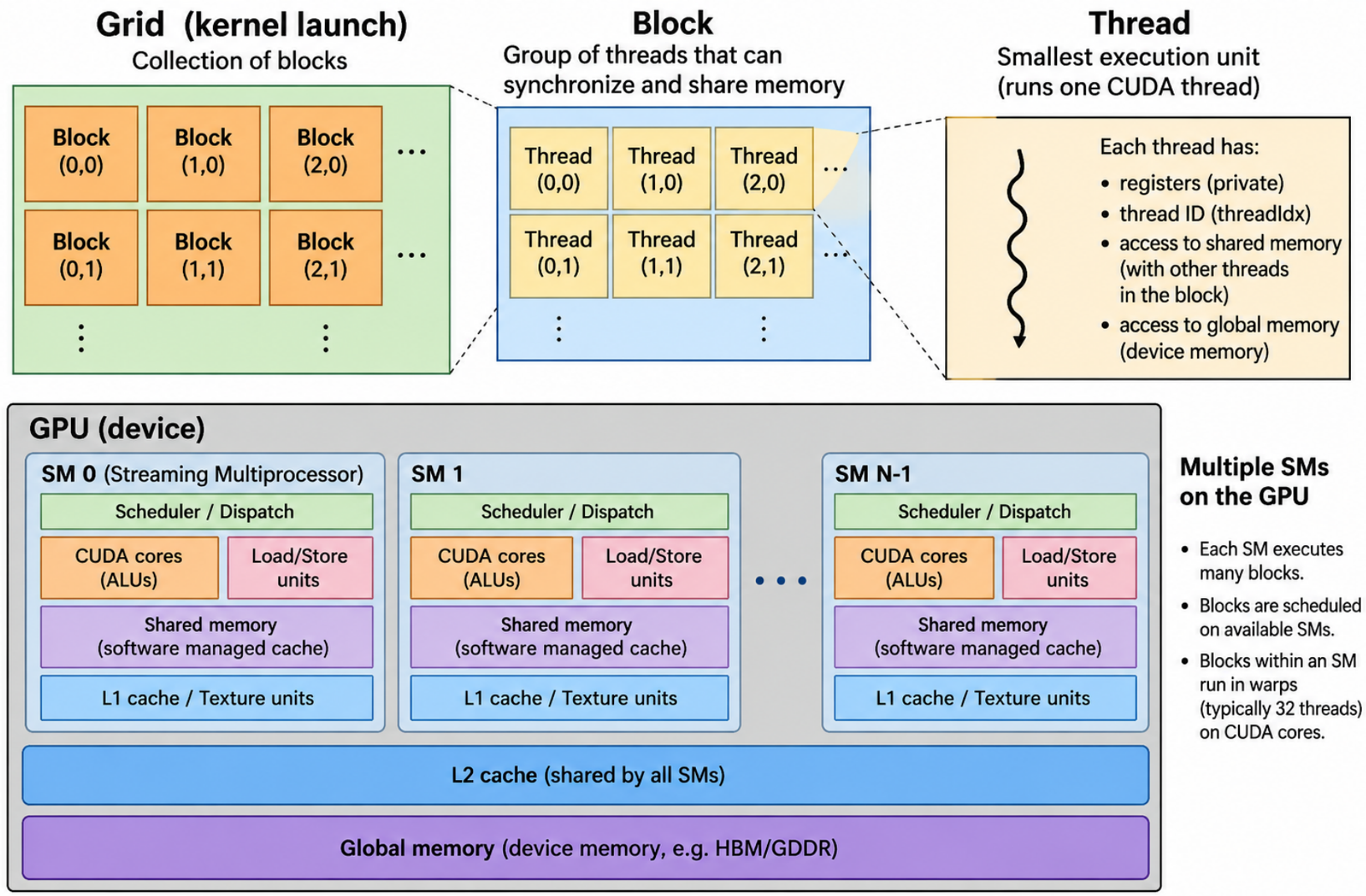
Speed

Throughput


GPU
CPU
SMEM
SMEM
SMEM
SMEM
Core
Core
Cache
PCIe Bus
Device (Global) Memory
(GDDRAM)
Host Memory
(DRAM)



GPU Architecture



- *threads* are grouped into *blocks*, and blocks are assigned to Streaming Multiprocessors (SMs). The collection of all blocks launched by a single CUDA kernel is *grid*.
- **Individual threads are not independently assigned to SMs.** The GPU schedules entire blocks onto SMs, and each SM executes the threads of its assigned blocks in groups called *warps* (typically by 32).

GPU programming

- Libraries (PyTorch, Tensorflow, JAX, cuFFT, cuBLAS, …)
  `$> pip install 'tensorflow[and-cuda]'`
- openACC (pragma's)
- CUDA
- Python libraries (cupy, polars, dask, numba)
- On SLURM
- to request GPU use `--gpus=1`, or `--gres='gpu:1'` on *gpu-xxx* partitions
  - `$> alloc -p gpu-ffa --gres "mps:5" --mem 10G -t 12:0:0 --cpus-per-task 2`
  - `--gres ‘gpu:1' # if you want entire GPU`
- `$> module load cuda/cuda-13.0`

openACC

- ```
  #pragma acc parallel
  {
  #pragma acc loop
  for (int i = 0; j < N; i++){
    #pragma atomic
    suma += a[i];
    }
  }
  ```

- ```
  $> module load nvidia-hpc-sdk
  $> module load nvhpc/25.9
  $> nvc -acc hello_acc.c -o hello_acc
  ```

- `-gpu=ccnative`

Data CPU ⇆ GPU

“Host” ⇆ “Target”

- ```
  #pragma acc data copyin(a[0:N], b[0:N])
  copyout(c[0:N])
  {
  #pragma acc parallel loop
  for (int i = 0; i < N; i++){
    c[i] = a[i] + b[i];
    }
  }
  ```

- `copy|copyin|copyout|create`
  ```

Gangs — workers — vectors

(aka grid — blocks — warps — threads)

- `#pragma acc parallel loop gang vector`

- `#pragma acc parallel num_gangs(80) num_workers(4) vector_length(32)`

```
{
    #pragma acc loop gang
    for (int i = 0; i < N; i++) {
        #pragma acc loop worker
        for (int j = 0; j < M; j++) {
            #pragma acc loop vector
            for (int k = 0; k < K; k++) {
                a[i][j][k] = b[i][j][k] + c[i][j][k]
            }
        }
    }
}
```

- There is no analog of `get_omp_thread_num()`, `#pragma omp critical`

openACC π

```
#include <stdio.h>
#include <stdlib.h>


int main(int argc, char **argv){
    long N = 100000000;
    double dx = 1.0 / N;
    double sum = 0.0;

    #pragma acc parallel loop reduction(+:sum)
    for (long i = 0; i < N; i++) {
        double x = (i + 0.5) * dx;
        sum += 4.0 / (1.0 + x*x);
    }
    double pi = dx * sum;
    return 0;
}
```

openACC hints

- Re-order loops or transpose arrays to enable SIMD/SIMT accesses in outermost loops [embarrassing parallelism]

- Use collapse(N) directives on loops to increase parallelism

- Replace critical sections with atomics

- Remove all I/O statements

- Remove memory allocation (on CPU)

- `$> nvc -O3 -acc -Minfo=accel|all|loop|par|inline|vect pi_acc.c -o pi_acc`

CUDA — Unleash the Force

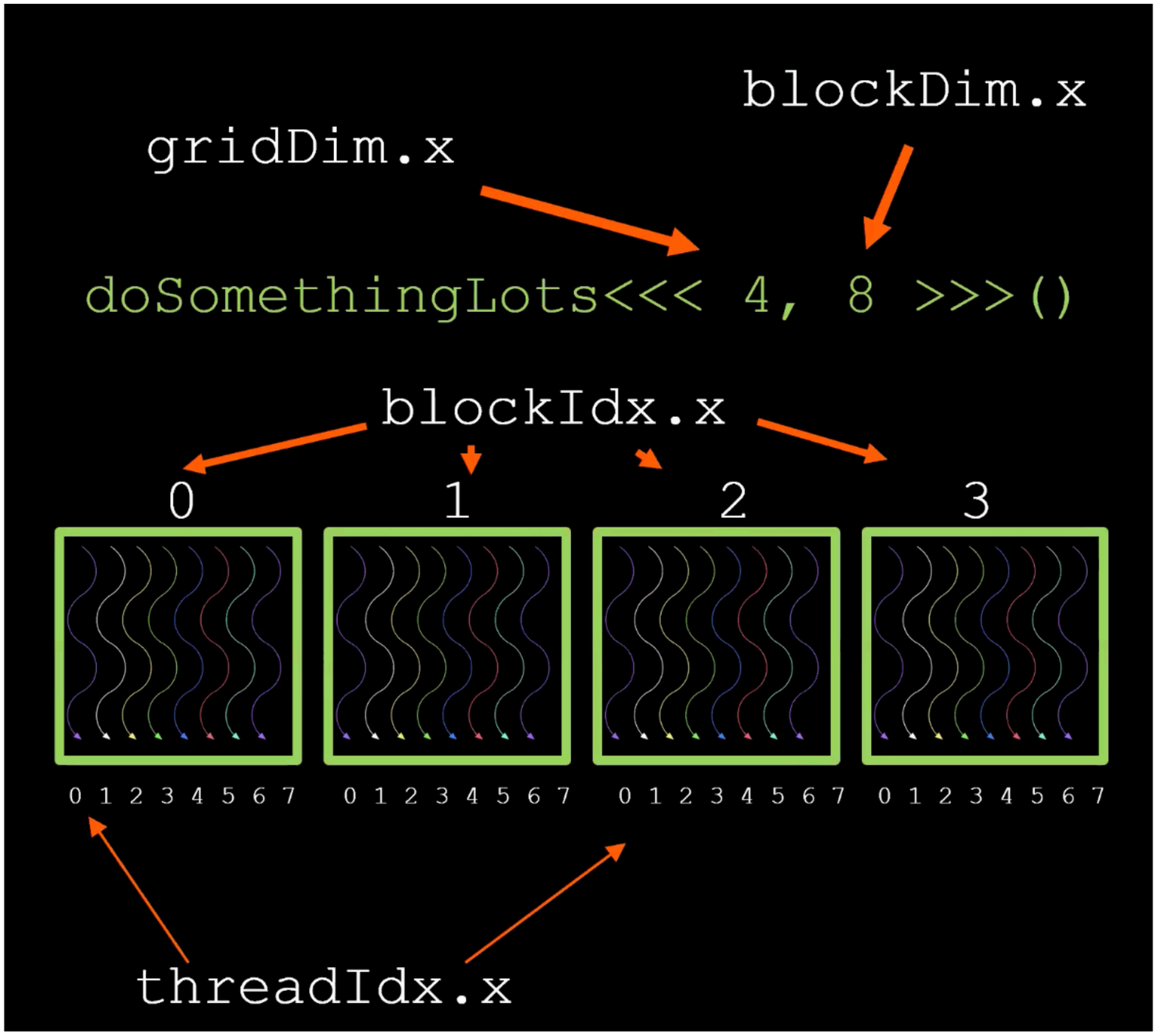
- CUDA threads:
  - Lightweight
  - Fast switching
  - > 10,000 execute simultaneously

```
__global__ void kernel( void) {
  printf("Hello from GPU!\n");
}
int main(void) {
kernel<<< 1,1 >>>();
cudaDeviceSynchronize();
return 0;
}
```

```
do_something<<< numBlocks, numThreads >>>(a, b, c);
```

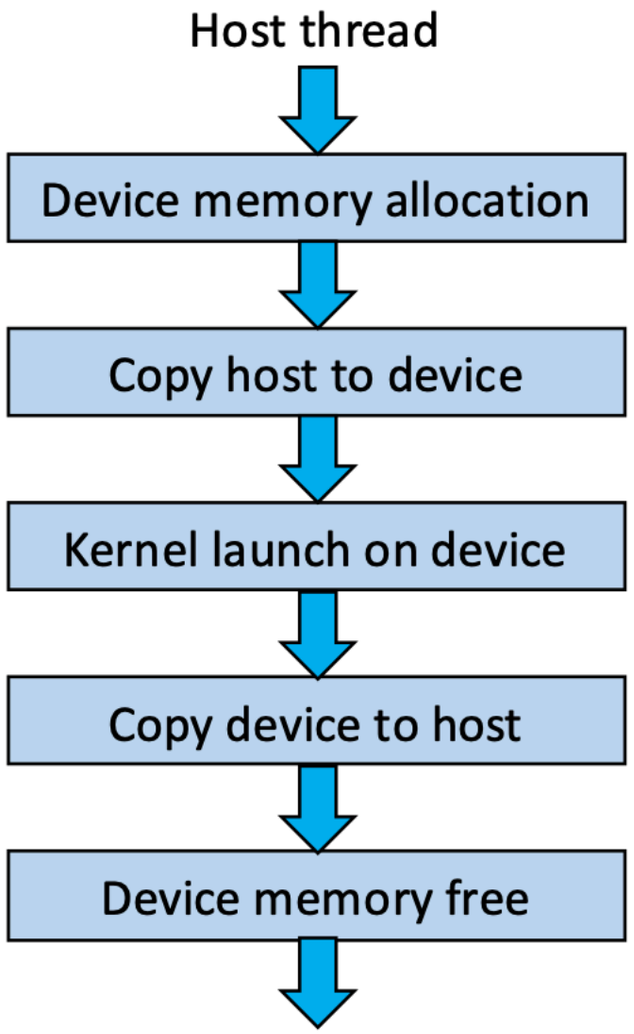
executes NumBlocks * NumThreads times in parallel
usually max. 1024 threads per block
warp = a group of 32 threads

Indexing threads



globalThreadID = threadIdx.x + blockIdx.x* blockDim.x

CUDA memory



`cudaMalloc()`

`cudaMemcpy()`

`my_kernel<<< … >>>()`

`cudaMemcpy()`

`cudaFree()`

```
cudaMalloc(&d_sum, sizeof(double));
cudaMallocManaged(
&sum, sizeof(double));
double *h_data =
(double*)malloc(N*sizeof(double))
cudaMemset(d_sum, 0, sizeof(double));
```

```
cudaMemcpy(
&h_sum, d_sum, sizeof(double),
cudaMemcpyDeviceToHost);
cudaMemcpy(
&d_sum, h_sum, sizeof(double),
cudaMemcpyHostToDevice);
```

N < total_num_threads

```
__global__ void square(float *a, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        a[i] = a[i] * a[i];
    }
}

… … …

int N = 1000;
int threadsPerBlock = 256;
int blocks = (N + threadsPerBlock - 1) / threadsPerBlock;
square<<<blocks, threadsPerBlock>>>(d_a, N);
```

N > total_num_threads
“grid-stride loop”

```
__global__ void square(float *a, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        a[i] = a[i] * a[i];
    }
}

… … …

square<<<4, 256>>>(d_a, 1000000);
```

Cuda π

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void calc_pi(double
*sum, int N){
    int i = blockIdx.x *
blockDim.x + threadIdx.x;

    if (i < N) {
        double x = (i + 0.5) / N;
        double value = 4.0 / (1.0
+ x*x);

        atomicAdd(sum, value);
    }
}
```

```
int main() {
    int N = 1000000;
    double *d_sum;
    double h_sum = 0.0;
    // Allocate GPU memory
    cudaMalloc(&d_sum,
sizeof(double));
    cudaMemset(d_sum, 0,
sizeof(double));
    // Configure CUDA execution
    int threadsPerBlock = 256;
    int blocks = (N + threadsPerBlock
- 1) / threadsPerBlock;
    // Launch kernel
    calc_pi<<<blocks,
threadsPerBlock>>>(d_sum, N);
    // Copy result back to CPU
    cudaMemcpy(&h_sum, d_sum,
                sizeof(double),
cudaMemcpyDeviceToHost);
    printf("Pi = %.15f\n", h_sum / N);
    cudaFree(d_sum);
    return 0;
}
```

CUDA with Python

- *numba*
- *cupy*
- *PyTorch*, *Tensorflow*, *polars*, *dask*, *Modulus* (PyTorch-based PINNs), *RAPIDS.ai* (*cuDF — pandas*-like, *cuML — scikit-learn*-like)…

Numba

- Just In Time (JIT) compiler both on CPU and on GPU
- `$> pip install numba numba-cuda`

```python
import numpy as np
from numba import vectorize

@vectorize
def saxpy(a, x, y) :
   return a*x+y

a = 3.141
X = np.random.rand(1024,2048)
y = np.random.rand(1024,2048)
result = saxpy(a, X, y)
```

```python
import numpy as np
from numba import vectorize

@vectorize([float32(float32,
float32, float32)],
target='cuda')
def saxpy(a, x, y) :
   return a*x+y

a = 3.141
x = np.random.rand(1024,2048)
y = np.random.rand(1024,2048)
result = saxpy(a, X, y)
```

cupy

- $> source ~/cupy-env/bin/activate
  $> module load cuda/13.0
  $> pip install cupy-cuda13

```
import numpy as np

def saxpy(a, x, y) :
   return a*x+y

a = 3.141
X = np.random.rand(1024,2048)
y = np.random.rand(1024,2048)
result = saxpy(a, X, y)
```

```
import cupy as cp

def saxpy(a, x, y) :
   return a*x+y

a = 3.141
× = cp.random.rand(1024,2048)
y = cp.random.rand(1024,2048)
result = saxpy(a, X, y)

print(cp.__version__)
print("GPU:",
cp.cuda.runtime.getDeviceProp
erties(0)["name"])
```

cupy π

- ```
  import cupy as cp
  import numpy as np

  N = 10_000_000

  # GPU array
  i = cp.arange(N, dtype=cp.float64)

  x = (i + 0.5) / N
  # Calculate integral on GPU
  pi = 4.0 * cp.sum(1.0 / (1.0 + x*x)) / N

  print("Pi =", pi.item()) # back to CPU
  ```

✅ GPU memory allocation

✅ CUDA kernel launches

✅ Parallel execution

✅ GPU reduction

a = np.arange(10) # CPU

b = cp.asarray(a) # copy to GPU
a = cp.asnumpy(b) # copy to CPU
  ```

Fuse kernels

```
kernel = cp.ElementwiseKernel(
    in_params,        # Input parameters
    out_params,       # Output parameters
    operation,        # CUDA C code
executed for each element
    name,             # Kernel name
)
```

```
import cupy as cp

pi_contribution = cp.ElementwiseKernel(
    'float64 i, float64 n',
    'float64 result',
    '''
    double x = (i + 0.5) / n;
    result = 4.0 / (1.0 + x*x) / n;
    ''',
    'pi_contribution'
)

N = 10_000_000
i = cp.arange(N, dtype=cp.float64)
values = pi_contribution(i, float(N))
pi = cp.sum(values)
print("Pi =", pi.item())
```

CuPy automatically provides the element index *i* inside the kernel:

```
values = pi_kernel(float(N), size=N)
```

Custom kernel

```
kernel = cp.RawKernel(
    code,       # CUDA source code
    name,       # Name of the kernel
func
    options=(),# Optional compiler
flags
)
```

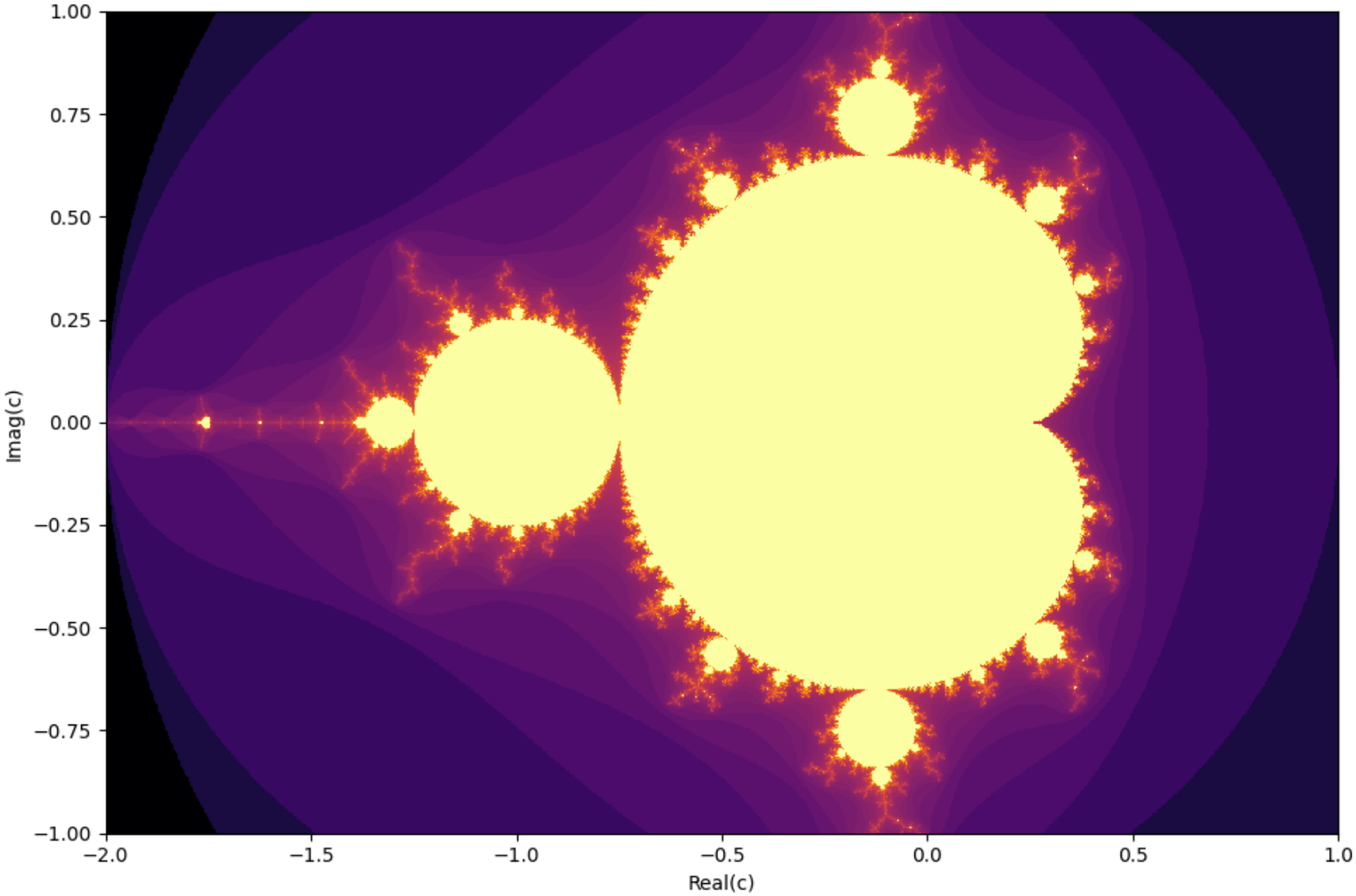
```
import cupy as cp

kernel = cp.RawKernel(r'''
extern "C" __global__
void square(const float* x, float* y, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
        y[i] = x[i] * x[i];
}
''', 'square')

n = 10
x = cp.arange(n, dtype=cp.float32)
y = cp.empty_like(x)
threads = 256
blocks = (n + threads - 1) // threads
kernel((blocks,), (threads,), (x, y, n))

print(y)
```

Mandelbrot Set



- In complex plain: $z \rightarrow z^2 + c$
- Escapes if $|z| > 2$ *or reaches* k_{max} iterations
- $z_0 = 0,\ c = 2$

Warm-up in benchmarking

```
def heavy_func(N):
  …
  return xyz
```

```
# warm-up
heavy_fce(10)

# serious run
start = time()
heavyfunc(10_000_000)
elapsed = time() - start
```

AI LLM — MCP

- Chimera-slurm: https://www.hpc.cuni.cz/chimera/ai-llm/

Ads: Quantum Computing

- Quantum Innovation Centre (QIC) + Czech Consortium in the IBM Quantum Network

- 3 years project launched 2025 (coordinated by CTU)

- Access to IBM quantum computers (400 mins/institution/ month) across Europe (399 qubits)

- Workshops, seminars

- Interested to learn beyond established projects?
  Contact Zdeňka Koupilová, zdenka.koupilova@matfyz.cuni.cz